

DIY Challenge Robot 2026

AprilTag EKF Calibration Guide

Equipment Required • Tag Printing & Surveying
Software Setup • Data Collection Procedure
Post-Processing & EKF Q/R Matrix Tuning
Validation & Results

ROS 2 Humble • FAST-LIO2 • robot_localization EKF • apriltag_ros

1 Overview & Purpose

AprilTag calibration is a **pre-competition, offline tuning session** run in your own test environment — not on the competition course. Its purpose is to provide centimetre-level ground-truth pose measurements so that the EKF (robot_localization) Q and R noise matrices can be tuned to match the real-world sensor behaviour of your specific robot.

The RealSense D435i camera detects printed AprilTag markers at known surveyed positions. The difference between the EKF-predicted pose and the tag-measured ground-truth pose (the *innovation sequence*) contains all information needed to tune the filter parameters automatically. Tighter, more accurate EKF fusion reduces localization drift per lap from a typical 10–20 cm to under 5 cm — critical for the 2-inch clearance narrow passages and movable-obstacle avoidance in the competition.

1.1 What This Session Gives You

Output	How It Helps Competition Performance
Per-section drift profile of FAST-LIO2	Reveals where odometry drifts most — tunnel exit, tight turns, gravel
Heading accuracy at low speed	RTK single-antenna heading is poor below ~0.3 m/s — tag data quantifies this
IMU contribution isolated by speed	Compare fast vs slow laps to separate IMU drift from scan-matching errors
Systematic error patterns	e.g. "always drifts left after a right turn" — fix in URDF or EKF config
Optimised EKF Q and R matrices	Saved to competition.yaml — 30–60 % drift reduction typical

■ NOTE: AprilTags are NOT present on the competition course. This is purely a pre-competition calibration tool used in your own test environment.

2 Equipment Required

2.1 AprilTag Hardware

Item	Specification	Where to Get	Notes
Printed AprilTag targets	8–10 tags, Tag36h11 family, IDs 0–9	Print free from april.eecs.umich.edu or generate with apriltag Python lib	Print on plain paper, mount on flat rigid board (foam board, cardboard). Minimum 20 × 20 cm per tag.
Rigid mounting boards	~40 × 40 cm, flat, non-reflective surface	Foam board / foam core from stationery shop	Prevents tag warping. Warp causes pose estimation error.
Ground stakes or stands	Anything that holds boards vertical and stable	Camera tripods, tent stakes, zip-tied to fence posts	Tags must face the robot path. Keep them at camera height (0.3–1.0 m).
RTK rover / base station	Your existing RTK GNSS setup	Already in hardware inventory	Used to survey exact (x, y, z) position of each tag centre to < 2 cm accuracy.
Measuring tape	5 m minimum	Hardware store	Backup measurement to cross-check RTK survey.

2.2 Robot Equipment

- **RealSense D435i** — already on robot, used for tag detection via RGB stream
- **Jetson Nano** — running full ROS 2 Humble stack
- **Hesai QT64** — running FAST-LIO2 for odometry during laps
- **RTK GNSS** — running for EKF2 global frame during laps
- **Joystick** — for driving laps (JOYSTICK mode, not autonomous)
- **Large SSD / USB drive** — rosbag from 10+ laps at full rate is 5–15 GB

■ **WARNING:** Do NOT use the D435i built-in IMU during this session. Use only the dedicated external IMU. USB latency jitter from the D435i causes timestamp drift that corrupts FAST-LIO2 fusion.

3 Software Setup

3.1 Install apriltag_ros

The repo references **christianrauch/apriltag_ros** — the standard ROS 2 Humble compatible package. Install via apt:

Install apriltag_ros

```
sudo apt update
sudo apt install ros-humble-apriltag-ros
```

```
# Verify
ros2 pkg prefix apriltag_ros
```

3.2 Camera Intrinsics Requirement

AprilTag pose estimation requires accurate camera intrinsics. If you have already run **scripts/calibrate_camera_intrinsics.sh** and your D435i intrinsics are stored, you are ready. If not, run camera intrinsics calibration first — see [Camera_Calibration_Guide.pdf](#).

Check camera info topic is publishing intrinsics

```
ros2 topic echo /camera/color/camera_info --once
```

3.3 Tag Configuration File

Create a tag configuration YAML that lists the tag family and sizes. Save as **src/challenge_bringup/config/apriltag_tags.yaml**:

src/challenge_bringup/config/apriltag_tags.yaml

```
tag_family: 36h11
tag_threads: 2
tag_decimate: 1.0
tag_blur: 0.0
tag_refine_edges: 1
tag_debug: 0
max_hamming_dist: 0
```

```
standalone_tags:
  - {id: 0, size: 0.20} # tag physical size in metres – adjust to your printout
  - {id: 1, size: 0.20}
  - {id: 2, size: 0.20}
  - {id: 3, size: 0.20}
  - {id: 4, size: 0.20}
  - {id: 5, size: 0.20}
  - {id: 6, size: 0.20}
  - {id: 7, size: 0.20}
  - {id: 8, size: 0.20}
  - {id: 9, size: 0.20}
```

■ NOTE: The "size" field is the length of one side of the black tag border in metres. Measure your printed tag — not the paper size. Incorrect size causes direct proportional error in the distance estimate.

3.4 Launch apriltag_ros with D435i

Run the apriltag_ros node pointed at the D435i colour stream. This can be done in a separate terminal alongside the main stack:

Terminal — run apriltag detection

```
# Source ROS and your workspace
```

```
source /opt/ros/humble/setup.bash
source install/setup.bash
```

```
ros2 run apriltag_ros apriltag_node --ros-args \
  -r image_rect:=/camera/color/image_raw \
  -r camera_info:=/camera/color/camera_info \
  --params-file src/challenge_bringup/config/apriltag_tags.yaml
```

Detections publish to: **/detections** (apriltag_ros/msg/AprilTagDetectionArray)

4 Physical Tag Placement & Surveying

4.1 Placement Rules

- Use **8–10 tags** spread around the entire test track perimeter
- Place tags so **at least 2 are visible** from any position on the track
- Face tags toward the inside of the track (toward the robot path)
- Height: tag centre at **0.3 m to 0.8 m** above ground — within D435i field of view
- Spacing: roughly every 3–5 m around the track
- Avoid placing tags in deep shadow or directly facing the sun
- Secure boards firmly — any movement between laps invalidates the ground truth

4.2 Surveying Tag Positions with RTK

Each tag must have its **centre position measured in the same world frame** that your EKF uses (typically UTM or local ENU origin set at robot start). Survey each tag with the RTK rover and record the values in a JSON file:

`calibration/tag_survey.json` (create manually after surveying)

```
{
  "frame_id": "map",
  "tags": [
    {"id": 0, "x": 0.00, "y": 0.00, "z": 0.50, "yaw_deg": 0.0},
    {"id": 1, "x": 5.20, "y": 0.15, "z": 0.50, "yaw_deg": 90.0},
    {"id": 2, "x": 10.45, "y": 0.10, "z": 0.50, "yaw_deg": 90.0},
    {"id": 3, "x": 10.50, "y": 8.30, "z": 0.50, "yaw_deg": 180.0},
    {"id": 4, "x": 5.10, "y": 8.25, "z": 0.50, "yaw_deg": 270.0},
    {"id": 5, "x": 0.05, "y": 8.20, "z": 0.50, "yaw_deg": 270.0},
    {"id": 6, "x": 2.50, "y": 4.10, "z": 0.50, "yaw_deg": 0.0},
    {"id": 7, "x": 7.80, "y": 4.00, "z": 0.50, "yaw_deg": 180.0},
    {"id": 8, "x": 1.20, "y": 2.50, "z": 0.50, "yaw_deg": 45.0},
    {"id": 9, "x": 9.10, "y": 6.50, "z": 0.50, "yaw_deg": 135.0}
  ]
}
```

■ NOTE: yaw_deg is the direction the tag **FACES** (i.e. the direction the robot must be approaching from to see it). 0° = facing +X axis (East). This is used only for documentation — `apriltag_ros` computes the full 6DOF pose from the image.

5 Data Collection Procedure

5.1 Pre-session Checklist

Check	Pass Condition
All tags firmly mounted and surveyed	tag_survey.json complete with all IDs
D435i publishing /camera/color/image_raw	ros2 topic hz shows ~30 Hz
apriltag_ros node running, /detections publishing	ros2 topic echo /detections shows tag IDs as you walk past
FAST-LIO2 running and converged	/odometry/fast_lio topic publishing, no drift warnings
RTK fix acquired	/fix topic shows status=2 (RTK fixed)
EKF1 and EKF2 running	ros2 node list shows ekf_local and ekf_global nodes
Rosbag recording started	See step 5.2
SSD has >20 GB free	df -h

5.2 Recording the Calibration Bag

Record a bag containing all topics needed for post-processing. Drive **10 or more laps** at competition speed with the joystick. Vary speed slightly between laps — slow laps isolate IMU drift, fast laps test scan-matching limits.

Record calibration bag

```
LABEL="apriltag_calib_$(date +%Y%m%d_%H%M%S)"  
mkdir -p calibration
```

```
ros2 bag record \  
  /tf \  
  /tf_static \  
  /odometry/fast_lio \  
  /odometry/filtered \  
  /odometry/global \  
  /fix \  
  /imu/data \  
  /detections \  
  /camera/color/camera_info \  
  --output calibration/${LABEL}
```

■ NOTE: Record /detections in addition to /tf — this is the raw apriltag_ros output containing the 6DOF pose of each detected tag relative to the camera.

5.3 During the Session

- Drive at **normal competition speed** (0.4–0.6 m/s) for the first 5 laps
- Drive 3 laps at **slow speed** (~0.15 m/s) to isolate IMU vs scan-matching drift
- Drive 2 laps at **fast speed** (~0.7 m/s) to test high-speed performance
- Do **not stop** mid-lap — continuous motion improves FAST-LIO2 accuracy
- Ensure each tag is within camera view range (**max ~3 m** for 20 cm tags) at some point each lap
- After all laps: stop recording, note the bag filename

✓ TIP: Save the pre-tuning `ekf_local.yaml` as `ekf_local_default.yaml` before overwriting, so you can revert if the tuned values perform worse.

7 tune_ekf_apriltag.py — Implementation Specification

This section is the complete specification for the post-processing script that needs to be written. One teammate should own this task.

7.1 Dependencies

`pip install dependencies`

`pip3 install rosbags numpy scipy matplotlib pyyaml transforms3d`

rosbags is used instead of `roscpp bag playback` — it reads bags offline without requiring a running ROS 2 node, which makes the script faster and runnable on any machine (laptop, not just Jetson).

7.2 Algorithm Steps

1	Load tag_survey.json Parse tag ground-truth positions into a dict: {tag_id: (x, y, z)} in map frame.
2	Open rosbag with rosbags library Read messages from /detections, /tf, /odometry/filtered topics.
3	Build TF tree from /tf and /tf_static Use the rosbags TF buffer to resolve transforms between frames at any timestamp.
4	For each /detections message Extract timestamp t , tag ID, and pose of tag relative to camera_link frame. Lookup robot pose (map $\rightarrow$ base_link) at time t from TF buffer. Transform tag detection into map frame using robot pose + camera_link transform. Compare to surveyed ground-truth position $\rightarrow$ compute innovation vector v_k .
5	Accumulate innovations per tag and per track section Group innovations by tag ID and by lap number. Track section = which tag was nearest when the observation was made.
6	Compute R_{optimal} $R_{\text{optimal}} = \text{mean}(v_k \cdot v_k^T)$ over all observations. Extract x, y, heading components for ekf_local.yaml.
7	Compute Q estimate Use Sage-Husa approximation or simple residual variance between /odometry/filtered and ground truth interpolated between tag observations.
8	Write output YAML Write calibration/ekf_tuning_results.yaml with new Q diagonal and R diagonal values, plus per-tag drift statistics (mean, std, max).
9	Plot drift profile Matplotlib figure: x-axis = track position (m along path), y-axis = EKF error (cm). Save as calibration/drift_plot.png.

7.3 Script Skeleton

scripts/tune_ekf_apriltag.py (skeleton to implement)

```
#!/usr/bin/env python3
"""
tune_ekf_apriltag.py – Post-process AprilTag calibration bag.
Usage:
    python3 scripts/tune_ekf_apriltag.py \
        --bag calibration/apriltag_calib_XXX \
        --survey calibration/tag_survey.json \
        --output calibration/ekf_tuning_results.yaml
"""
import argparse, json, yaml, numpy as np, matplotlib.pyplot as plt
from pathlib import Path
from rosbags.rosbag2 import Reader
from rosbags.typesys import get_types_from_msg, register_types

def load_survey(path):
    """Return dict {tag_id: np.array([x, y, z])}"""
    with open(path) as f:
        data = json.load(f)
    return {t["id"]: np.array([t["x"], t["y"], t["z"]]) for t in data["tags"]}

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--bag", required=True)
    ap.add_argument("--survey", required=True)
    ap.add_argument("--output", default="calibration/ekf_tuning_results.yaml")
    args = ap.parse_args()

    survey = load_survey(args.survey)
    innovations = [] # list of np.array([dx, dy, dheading])

    with Reader(args.bag) as reader:
        # TODO: iterate messages, build TF buffer,
        #       extract detections, compute innovations
        pass

    innovations = np.array(innovations) # shape (N, 3)
    R_optimal = np.mean(innovations[:, :, None] * innovations[:, None, :], axis=0)

    results = {
        "R_diagonal_xy_heading": R_optimal.diagonal().tolist(),
        "num_observations": len(innovations),
        "mean_error_m": float(np.mean(np.linalg.norm(innovations[:, :2], axis=1))),
        "max_error_m": float(np.max(np.linalg.norm(innovations[:, :2], axis=1))),
    }
    with open(args.output, "w") as f:
        yaml.dump(results, f)
    print(f"Results written to {args.output}")
    print(f"Mean position error: {results['mean_error_m']*100:.1f} cm")

if __name__ == "__main__":
    main()
```

8 Validation After Tuning

8.1 Validation Procedure

After applying the tuned EKF parameters, run **5 fresh validation laps** (not used in the tuning data) and re-run the post-processing script. The mean error should decrease by 30–60 % compared to the pre-tuning baseline.

Metric	Target (Pre-Tuning Typical)	Target (Post-Tuning)
Mean position error per lap	10–20 cm	< 5 cm
Max position error per lap	30–50 cm	< 15 cm
Heading error at tag	2–5°	< 1.5°
Lap-2 drift vs Lap-1	5–15 cm	< 5 cm
Narrow passage lateral error	4–8 cm	< 4 cm (2" clearance budget)

8.2 If Results Are Worse After Tuning

- Check that the tag survey measurements are accurate — re-measure with RTK
- Check the tag_survey.json frame_id matches the EKF map frame origin
- Ensure apriltag_ros is using the correct tag size (measure your printout physically)
- Revert to ekf_local_default.yaml and try tuning with only 5 laps (reduce overfitting)
- Check camera intrinsics are calibrated — bad intrinsics = bad tag pose estimates

8.3 Save Results

Archive tuning results with git

```
git add calibration/tag_survey.json \  
      calibration/ekf_tuning_results.yaml \  
      src/diy_localization/config/ekf_local.yaml  
git commit -m "calib: AprilTag EKF tuning session YYYYMMDD"
```

■ NOTE: Do NOT commit the rosbag to git — it is too large. Store it on an external SSD and reference it in calibration/ekf_tuning_results.yaml under a "source_bag" key.

9 Quick Reference — Day-of Checklist

#	Action	Command / Notes
1	Print and mount 8–10 Tag36h11 tags	Min 20×20 cm, flat board, camera-height
2	Survey each tag centre with RTK	Record to calibration/tag_survey.json
3	Source workspace	source /opt/ros/humble/setup.bash && source install/setup.bash
4	Launch full robot stack	bash scripts/run_robot.sh laptop
5	Launch apriltag_ros	ros2 run apriltag_ros apriltag_node (see §3.4)
6	Verify /detections	ros2 topic echo /detections — walk past a tag to confirm
7	Start bag recording	ros2 bag record /tf /tf_static /odometry/fast_lio /odometry/filtered /detections /fix
8	Drive 10+ laps	Mix of normal, slow, fast speed laps
9	Stop recording, note bag name	
10	Run post-processing	python3 scripts/tune_ekf_apriltag.py --bag ... --survey ... (§6.3)
11	Apply results to ekf_local.yaml	See §6.4
12	Run 5 validation laps + re-process	Confirm improvement — see §8.1
13	Commit results to git	See §8.3